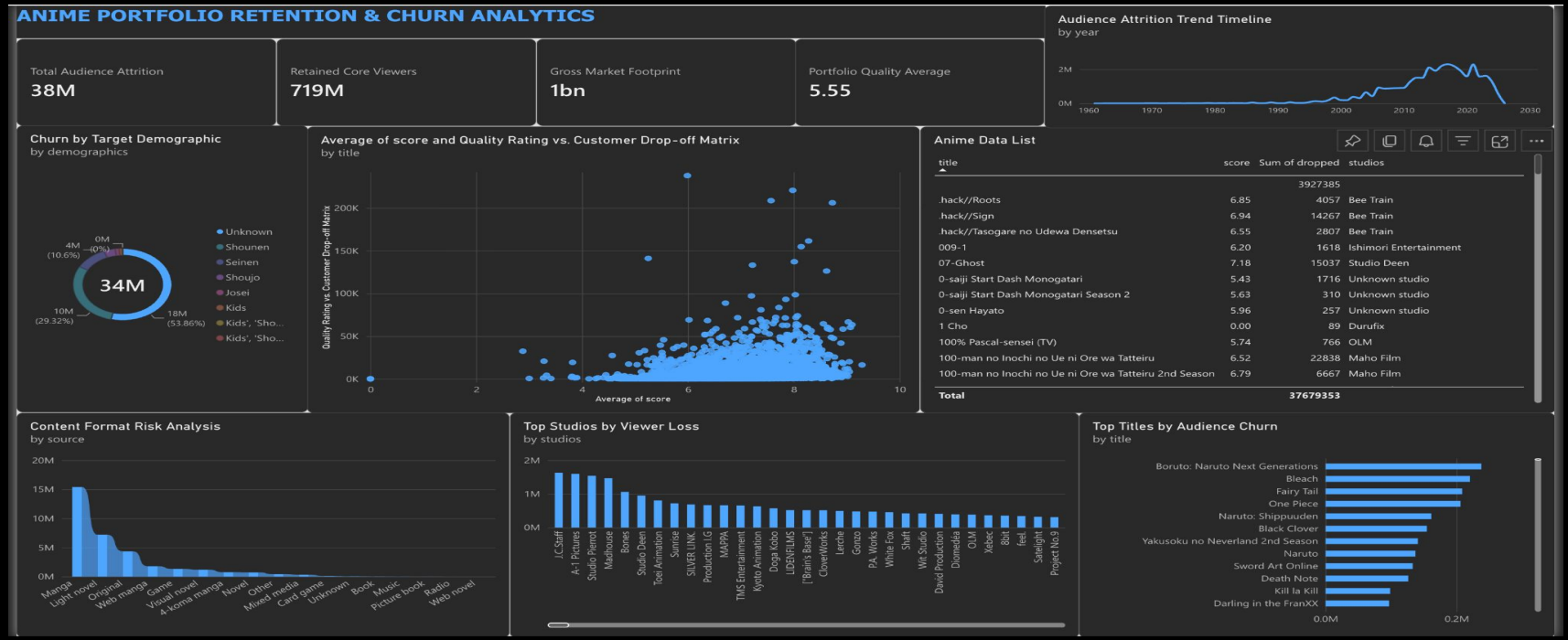
ANIME PORTFOLIO RETENTION & CHURN ANALYTICS

- **The Business Idea:** Track audience drop-off and score patterns across different anime genres and studios to help streaming platforms optimize content licensing and retain subscribers.
- **What I Did:** I built an interactive **Anime Portfolio Retention & Churn Analytics** dashboard that displays total viewer attrition metrics, demographic breakdowns, studio loss trends, and a custom score-versus-quality drop-off matrix tracking a **38M total audience attrition**.



Data Pipeline & Warehouse Setup (Python & SQL)

- **What I Did:** I engineered an automated data pipeline that extracts audience behavior logs, handles text data cleaning in Pandas, and structures a relational SQL schema.
- **Database Engineering:** I wrote SQL scripts inside Python to create an official **Star Schema**—building a Dimension table (`dim_anime_details`) and a Fact table (`fact_anime_stats`) linked by a primary key (`mal_id`) and optimized with custom database indexing.

```python
import pandas as pd

print("Reading details.csv.")
df_details = pd.read_csv("details.csv")[[
    'mal_id', 'title', 'type', 'status', 'genres', 'studios',
    'rating', 'episodes', 'year', 'season', 'demographics', 'source',
    'score', 'scored_by'
]]

print("Cleaning fields.")
for col in ['title', 'type', 'status', 'genres', 'studios', 'rating', 'season', 'demographics', 'sourc
    df_details[col] = df_details[col].fillna('Unknown').str.strip()

df_details['mal_id'] = pd.to_numeric(df_details['mal_id'], errors='coerce').fillna(0).astype(int)
df_details['year'] = pd.to_numeric(df_details['year'], errors='coerce').fillna(0).astype(int)
df_details['episodes'] = pd.to_numeric(df_details['episodes'], errors='coerce').fillna(12).astype(int)
df_details['score'] = pd.to_numeric(df_details['score'], errors='coerce').fillna(0.0)
df_details['scored_by'] = pd.to_numeric(df_details['scored_by'], errors='coerce').fillna(0).astype(int

print("Cleaning stats.csv.")
df_stats = pd.read_csv("stats.csv")[['mal_id', 'completed', 'dropped', 'total']]

for col in ['mal_id', 'completed', 'dropped', 'total']:
    df_stats[col] = pd.to_numeric(df_stats[col], errors='coerce').fillna(0).astype(int)

print("Saving files")
df_details.to_csv("clean_details.csv", index=False)
df_stats.to_csv("clean_stats.csv", index=False)

print("Success!")
```

```python
import sqlite3
import pandas as pd

print("Connecting to local SQL Engine.")
conn = sqlite3.connect("anime_warehouse.db")
cursor = conn.cursor()
print("Star Schema")

cursor.executescript("""
DROP TABLE IF EXISTS dim_anime_details;
DROP TABLE IF EXISTS fact_anime_stats;

CREATE TABLE dim_anime_details (
    mal_id INTEGER PRIMARY KEY,
    title TEXT,
    type TEXT,
    status TEXT,
    genres TEXT,
    studios TEXT,
    rating TEXT,
    episodes INTEGER,
    year INTEGER,
    season TEXT,
    demographics TEXT,
    source TEXT,
    score REAL,
    scored_by INTEGER
);

CREATE TABLE fact_anime_stats (    mal_id INTEGER PRIMARY KEY,
    completed INTEGER,
    dropped INTEGER,
    total INTEGER,
    FOREIGN KEY (mal_id) REFERENCES dim_anime_details(mal_id)
);
""")

df_details = pd.read_csv("clean_details.csv")
df_stats = pd.read_csv("clean_stats.csv")

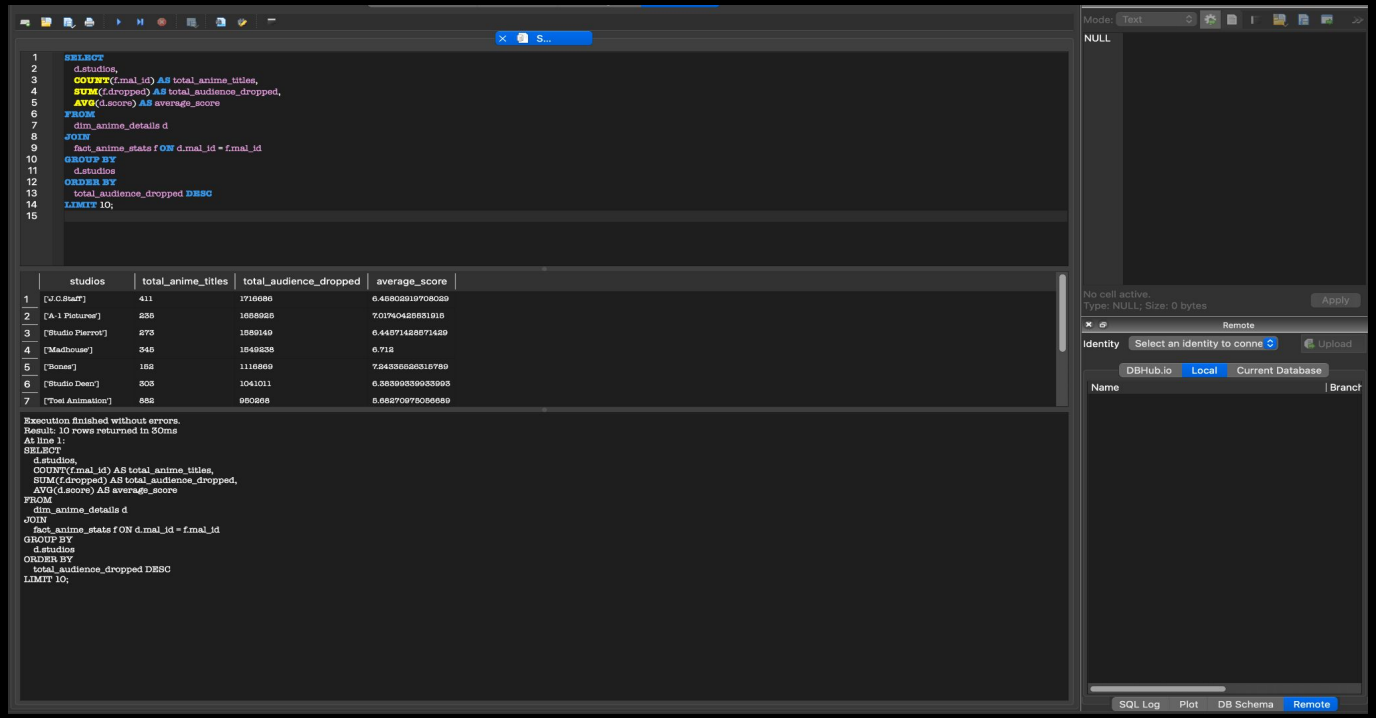
# Append and insert data records straight into the active relational SQL tables
df_details.to_sql("dim_anime_details", conn, if_exists="append", index=False)
df_stats.to_sql("fact_anime_stats", conn, if_exists="append", index=False)
print("Indexing database relationship parameters.")

cursor.execute("CREATE INDEX IF NOT EXISTS idx_details_id ON dim_anime_details(mal_id);")
cursor.execute("CREATE INDEX IF NOT EXISTS idx_stats_id ON fact_anime_stats(mal_id);")

conn.commit()
conn.close()
```

Data Pipeline & Warehouse Setup (Python & SQL)

- **Database Engineering:** I wrote SQL scripts inside Python to create an official **Star Schema**—building a Dimension table (`dim_anime_details`) and a Fact table (`fact_anime_stats`) linked by a primary key (`mal_id`) and optimized with custom database indexing.



Business Problem Solved & Power BI Impact

The Core Problem: Platforms lose massive viewer engagement because they cannot pinpoint if audience churn is driven by poor content scores, target demographic mismatches, or specific content formats.

The Solution: I built an interactive score-versus-quality drop-off matrix card linked to title-by-title data list ledgers, allowing teams to isolate exactly which titles or studios are driving user drop-offs.

What It Solved Better: It replaced static tracking sheets with an automated command center, giving content managers an instant view of historical churn to pull out of low-performing studio portfolios before losing revenue.

Key Learnings: I learned how to model an optimal star schema database design from scratch, proving that partitioning large datasets into structured dimensions and facts dramatically speeds up live cross-filtering in a reporting canvas.